

Phase 2 Feasibility Report

TraceGraph: Revisitable Memory Agents with Graph-Based Memory Traversal for Multi-Hop Long-Context Reasoning

CECS 551 Term Project – Phase 2

Team 8: Utkarsh Balu Lubal | Kashif Manzer | Mayank Maurya | Dipak Yadav

1 Market Research: Existing Solutions and Gaps

Market need. Organizations increasingly deploy Retrieval-Augmented Generation (RAG) to answer questions over internal documentation (policies, wikis, PRDs, SOPs, meeting notes), customer support knowledge bases, and research corpora. High-stakes domains (compliance, legal, security, finance, operations) require **high-confidence answers** with **auditability** and clear provenance.

Representative existing solutions.

- **Flat top- k RAG frameworks:** LangChain retrieval workflows and retrievers typically return top- k passages from vector stores/indices and then synthesize an answer [5]. LlamaIndex similarly describes RAG as indexing data and filtering it down to the most relevant context for the LLM [6].
- **GraphRAG approaches:** Microsoft GraphRAG presents a structured approach to RAG using LLM-derived graphs/hierarchical context building rather than naive snippet retrieval [1, 2]. Neo4j provides a GraphRAG package and describes GraphRAG as improving answer quality and explainability using graph structure [3, 4].

Gap analysis (what still fails in practice).

- **Multi-hop retrieval failure:** Flat top- k retrieval often misses evidence chains distributed across documents, causing incomplete context and hallucinations.
- **Weak traceability:** Citations show *where* text came from, but not *how* evidence pieces connect into a reasoning chain.
- **Underspecified queries:** Many user questions require iterative refinement (discover prerequisites, disambiguate entities) rather than one-shot retrieval.
- **Cost/scale friction:** Context-window expansion is expensive; naive all-pairs linking can be quadratic and hard to maintain as corpora grow.

Feasibility hypothesis. A revisitable agent paired with **bounded graph traversal retrieval** can address these gaps by (i) discovering multi-hop evidence chains and (ii) explicitly reporting retrieval paths for auditability.

2 AI Technique Selection (Technical Feasibility)

We select techniques that are implementable within the course timeline and directly address the identified gaps:

- (1) **LLM + RAG (generation grounded in retrieved evidence).** Use an LLM for response synthesis with citation-first prompting to reduce hallucinations.
- (2) **NLP metadata extraction for graph construction.** Use spaCy for named entity extraction and TF-IDF/BM25 for lexical signals to build node metadata and edges (NEXT adjacency, SHARED_ENTITY overlap, LEXICAL_SIM).
- (3) **Graph-based retrieval via bounded traversal.** Perform seed selection using indexed retrieval (BM25/entity lookup), then expand using bounded beam search / priority-queue BFS to retrieve a compact evidence subgraph that supports multi-hop reasoning.
- (4) **Revisitable (iterative) agent loop.** Allow multiple retrieval “callbacks”: draft partial answer → detect missing prerequisite → refine query → traverse again → finalize with citations and a retrieval-path explanation.

Complexity/scalability rationale. We avoid $O(n^2)$ all-pairs comparisons by using indexed retrieval plus a sparse, bounded-degree graph and budgeted traversal. Practically, indexing/sparse construction behaves closer to $O(n \log n)$ and query-time cost is controlled by traversal budgets.

3 Data Plan (Dataset Usage / Open Data Strategy)

Primary evaluation datasets (multi-hop QA).

- **HotpotQA:** multi-hop QA with supporting facts, enabling evidence-coverage evaluation.
- **2WikiMultiHopQA:** multi-hop QA over Wikipedia articles for robustness across a second benchmark.

Data preparation and usage.

- Treat each Wikipedia article (or provided context) as a document; chunk into nodes; build the memory graph with metadata and edges.
- Use dataset questions as queries; compare baselines vs TraceGraph retrieval.

Baselines and evaluation metrics.

- **Baselines:** BM25 top- k + LLM; embedding top- k + LLM (optional); traversal without revisitable callbacks.
- **Metrics:** answer EM/F1; **evidence coverage proxy** (gold supporting facts present in retrieved context); efficiency (token usage, latency); qualitative case studies showing retrieval paths.

Open-data demo corpus (optional). In addition to benchmarks, we will demonstrate on an open or non-sensitive corpus (public policies, public wiki, or course-like public materials) to showcase enterprise-style “why” questions with retrieval-path explanations.

Phase 2 compliance: This report includes (1) market research & existing solutions with gaps, (2) selected AI techniques, and (3) a data plan, matching the Phase 2 deliverable requirements.

References

- [1] Microsoft GraphRAG (project site). <https://microsoft.github.io/graphrag/>
- [2] microsoft/graphrag (GitHub repository). <https://github.com/microsoft/graphrag>
- [3] Neo4j GraphRAG Python package (GitHub). <https://github.com/neo4j/neo4j-graphrag-python>
- [4] Neo4j: “What is GraphRAG?” <https://neo4j.com/blog/genai/what-is-graphrag/>
- [5] LangChain docs: Retrieval. <https://docs.langchain.com/oss/python/langchain/retrieval>
- [6] LlamaIndex docs: Introduction to RAG. <https://developers.llamaindex.ai/python/framework/understanding/rag/>